

uniApp 扩展组件 uni-ui 基本使用方法（Vue3 setup版）

uni-ui 是 DCloud 官方推出的基于 uni-app 的高性能 UI 组件库，支持多端适配（小程序、H5、App），提供了丰富的组件（基础、表单、布局、导航等），可快速弥补官方原生控件的不足，简化开发流程、提升页面美观度。以下讲解其核心安装配置、常用组件使用方法，全程适配 Vue3 setup 语法，兼顾实用性和易用性。

一、uni-ui 前置准备（安装+配置）

uni-ui 不属于 uniApp 内置控件，使用前需先安装并配置，支持两种主流安装方式，推荐新手使用 uni_modules 安装（无需额外配置，自动导入）。

1. 安装方式（两种可选）

方式一：uni_modules 安装（推荐，HBuilderX 专用）

适用于 HBuilderX 开发的 uniApp 项目，无需配置 easycom，组件自动注册，无需手动引入。

1. 打开 HBuilderX，选中当前 uniApp 项目，右键点击项目根目录；
2. 选择「从插件市场导入」，在搜索框中输入「uni-ui」，找到官方发布的「uni-ui 组件库」；
3. 点击「导入」，等待安装完成，安装后项目根目录会新增「uni_modules」文件夹，包含所有 uni-ui 组件。

方式二：NPM 安装（适用于 Vue-CLI 或非 HBuilderX 项目）

1. 打开终端，进入项目根目录，执行安装命令：

```
npm install @dcloudio/uni-ui --save
```

2. 安装完成后，需在项目根目录的 pages.json 中配置 easycom，实现组件自动导入（无需手动 import）：

```
{
  "easycom": {
    "autoscan": true,
    "custom": {
      "^uni-(.*)": "@dcloudio/uni-ui/lib/uni-$1/uni-$1.vue"
    }
  },
  "pages": [
    // 你的页面配置
  ]
}
```

3. 若项目使用 Sass，需额外安装依赖（uni-ui 部分组件依赖 Sass）：

```
npm install sass sass-loader@10.1.1 -D
```

2. 配置说明

两种安装方式完成后，均无需手动引入组件（除非关闭 easycom 自动导入），直接在 template 中使用 uni-ui 组件即可，Vue3 setup 语法下无需额外注册组件。

若项目使用 TypeScript，需在 tsconfig.json 中添加类型定义，获得更好的代码提示：

```
{
  "compilerOptions": {
    "types": [
      "@dcloudio/types",
      "miniprogram-api-typings",
      "@uni-helper/uni-app-types",
      "@uni-helper/uni-ui-types"
    ]
  }
}
```

二、uni-ui 核心常用组件使用方法

uni-ui 组件分为基础组件、表单组件、布局组件、导航组件等类别，以下选取开发中最高频的组件，结合 Vue3 setup 语法讲解，每个组件均提供完整示例代码，可直接复制使用。

1. 基础组件：uni-button（增强型按钮）

基于 uniApp 原生 button 扩展，支持更多样式、图标、加载状态，多端适配性更强，适用于表单提交、操作触发等场景。

```
1  <template>;
2    <view class="btn-container">;
3      <!-- 基础样式按钮 -->
4      <uni-button type="primary" @click="handleClick">主要按钮</uni-button>
5      <uni-button type="default" @click="handleClick">默认按钮</uni-button>
6      <uni-button type="warn" @click="handleDelete">警告按钮</uni-
button>;
7
8      <!-- 带图标按钮 -->
9      <uni-button type="primary" icon="search">搜索按钮</uni-button>;
10
11     <!-- 加载状态按钮 -->
12     <uni-button type="primary" :loading="isLoading" @click="handleLoad">加载中
</uni-button>;
13
14     <!-- 禁用按钮 -->
15     <uni-button type="primary" disabled>禁用按钮</uni-button>
16
17     <!-- 自定义尺寸按钮 -->
18     <uni-button type="primary" style="width: 200rpx; height: 80rpx;">小按钮
</uni-button>
19   </view>
20 </template>
21
22 <script setup>
23 import { ref } from 'vue'
24
25 const isLoading = ref(false)
26
27 // 按钮点击事件
28 const handleClick = () => {
29   uni.showToast({ title: '点击成功', icon: 'success' })
30 }
31
32 // 警告按钮点击（删除操作）
33 const handleDelete = () => {
34   uni.showModal({
35     title: '提示',
36     content: '确定要删除吗？',
37     success: (res) => {
38       if (res.confirm) {
39         uni.showToast({ title: '删除成功', icon: 'success' })
40       }
41     }
42   })
43 }
```

```

41     }
42   })
43 }
44
45 // 加载状态切换
46 const handleLoad = () => {
47   isLoading.value = true
48   setTimeout(() => {
49     isLoading.value = false
50   }, 2000)
51 }
52 </script>
53
54 <style scoped>
55 .btn-container {
56   padding: 30rpx;
57   display: flex;
58   flex-direction: column;
59   gap: 20rpx;
60 }
61 </style>

```

2. 基础组件：uni-badge（数字角标）

用于展示未读消息数量、状态标记等场景，支持多种样式、颜色，可嵌套在按钮、图标、文本旁使用，是消息通知的常用组件。

```

1  <template>
2    <view class="badge-container">
3      <!-- 基础角标（右上角） -->
4      <view class="badge-item">
5        <uni-button type="primary">
6          消息
7          <uni-badge text="5"></uni-badge>
8        </uni-button>
9      </view>
10
11     <!-- 不同颜色角标 -->
12     <view class="badge-item">
13       <uni-icon type="comment" size="32"></uni-icon>
14       <uni-badge text="10" type="success"></uni-badge>
15     </view>
16
17     <!-- 红点角标（无数字） -->
18     <view class="badge-item">

```

```

19      <text class="text">通知</text>
20      <uni-badge type="warn" dot></uni-badge>
21    </view>;
22
23    <!-- 自定义颜色角标 -->
24    <view class="badge-item">
25      <uni-badge text="新" color="#fff" bgColor="#67c23a"></uni-badge>
26    </view>
27  </view>
28 </template>
29
30 <script setup></script>
31
32 <style scoped>
33 .badge-container {
34   padding: 30rpx;
35   display: flex;
36   gap: 30rpx;
37   align-items: center;
38   flex-wrap: wrap;
39 }
40 .badge-item {
41   display: flex;
42   align-items: center;
43   position: relative;
44 }
45 .text {
46   font-size: 32rpx;
47   color: #333;
48 }
49 </style>

```

3. 表单组件：uni-easyinput（增强型输入框）

基于原生 input 扩展，内置清除、密码显示/隐藏、输入验证等功能，无需额外编写逻辑，适用于账号、密码、搜索等表单输入场景。

```

1  <template>
2    <view class="input-container">
3      <!-- 普通文本输入（带清除按钮） -->
4      <uni-easyinput
5        v-model="username"
6        placeholder="请输入账号"
7        clearable
8        @input="handleInput"

```

```
9      />
10
11    <!-- 密码输入（带显示/隐藏切换） -->
12    <uni-easyinput
13      v-model="password"
14      placeholder="请输入密码"
15      type="password"
16      password-icon
17      style="margin-top: 20rpx;"
18    />
19
20    <!-- 搜索输入框（带搜索图标） -->
21    <uni-easyinput
22      v-model="search"
23      placeholder="请输入搜索内容"
24      search-icon
25      style="margin-top: 20rpx;"
26      @confirm="handleSearch"
27    />;
28
29    <!-- 禁用状态输入框 -->
30    <uni-easyinput
31      v-model="disabledInput"
32      placeholder="禁用输入框"
33      disabled
34      style="margin-top: 20rpx;"
35    />
36  </view>
37 </template>
38
39 <script setup>
40 import { ref } from 'vue'
41
42 // 响应式输入值
43 const username = ref('')
44 const password = ref('')
45 const search = ref('')
46 const disabledInput = ref('不可编辑')
47
48 // 输入监听
49 const handleInput = (e) => {
50   console.log('账号输入: ', e.detail.value)
51 }
52
53 // 搜索确认
54 const handleSearch = () => {
55   if (!search.value) {
```

```

56     uni.showToast({ title: '请输入搜索内容', icon: 'none' })
57     return
58   }
59   uni.showToast({ title: `搜索: ${search.value}`, icon: 'none' })
60 }
61 </script>
62
63 <style scoped>
64   .input-container {
65     padding: 30rpx;
66   }
67   uni-easyinput {
68     --input-height: 80rpx;
69     --input-font-size: 30rpx;
70   }
71 </style>

```

4. 布局组件：uni-list + uni-list-item（列表组件）

uni-ui 封装的列表组件，自带统一样式，支持图标、副标题、箭头、开关等，无需手动编写列表样式，适用于个人中心、消息列表、设置页面等场景。

```

1  <template>
2    <view class="list-container">
3      <uni-list>
4        <!-- 基础列表项 -->
5        <uni-list-item title="我的订单" @click="goToOrder">&lt;/uni-list-
item>
6
7        <!-- 带图标的列表项 -->
8        <uni-list-item title="我的收藏" icon="star" @click="goToCollect"></uni-
list-item>
9
10       <!-- 带副标题的列表项 -->
11       <uni-list-item title="联系客服" subtitle="工作时间：9:00-18:00"
icon="phone"></uni-list-item>
12
13       <!-- 带箭头的列表项（跳转提示） -->
14       <uni-list-item title="设置" icon="setting" right-icon="arrowright"
@click="goToSetting"></uni-list-item>
15
16       <!-- 带开关的列表项 -->
17       <uni-list-item title="消息通知">
18         <template #right>

```

```

19         <uni-switch v-model="switchValue" @change="handleSwitch"></uni-
switch>
20     </template>
21 </uni-list-item>
22 </uni-list>
23 </view>
24 </template>
25
26 <script setup>
27 import { ref } from 'vue'
28 import { router } from '@/utils/router.js' // 可复用之前封装的路由
29
30 const switchValue = ref(true)
31
32 // 列表项点击事件
33 const goToOrder = () => {
34     router.push('/pages/order/order')
35 }
36 const goToCollect = () => {
37     router.push('/pages/collect/collect')
38 }
39 const goToSetting = () => {
40     router.push('/pages/setting/setting')
41 }
42
43 // 开关切换事件
44 const handleSwitch = (e) => {
45     console.log('消息通知状态: ', e.detail.value)
46     switchValue.value = e.detail.value
47 }
48 </script>
49
50 <style scoped>
51 .list-container {
52     padding: 20rpx 0;
53 }
54 uni-list-item {
55     --list-item-height: 100rpx;
56     --list-item-title-font-size: 30rpx;
57 }
58 </style>

```

5. 布局组件：uni-grid + uni-grid-item（宫格组件）

用于实现九宫格布局，适用于功能入口、分类展示等场景，支持自定义列数、图标、文本，自带适配样式，无需手动处理布局对齐。


```
1  <template>
2    <view class="grid-container">
3      <uni-grid column-num="3" border="false">
4        <uni-grid-item v-for="(item, index) in gridList" :key="index"
@click="handleGridClick(item)">
5          <view class="grid-icon">
6            <uni-icon :type="item.icon" size="40" color="#409eff"></uni-icon>
7          </view>
8          <view class="grid-text">{{ item.text }}</view>
9        </uni-grid-item>
10     </uni-grid>
11   </view>
12 </template>
13
14 <script setup>
15 import { ref } from 'vue'
16
17 // 宫格数据源
18 const gridList = ref([
19   { icon: 'home', text: '首页' },
20   { icon: 'cart', text: '购物车' },
21   { icon: 'orders-o', text: '订单' },
22   { icon: 'user', text: '我的' },
23   { icon: 'coupon', text: '优惠券' },
24   { icon: 'help', text: '帮助中心' }
25 ])
26
27 // 宫格点击事件
28 const handleGridClick = (item) => {
29   uni.showToast({ title: `点击${item.text}`, icon: 'none' })
30 }
31 </script>
32
33 <style scoped>
34 .grid-container {
35   padding: 30rpx;
36 }
37 .grid-icon {
38   margin-bottom: 15rpx;
39   display: flex;
40   justify-content: center;
41 }
42 .grid-text {
43   font-size: 28rpx;
44   color: #333;
45   text-align: center;
```

```
46   }
47   </style>
```

6. 交互组件：uni-popup（弹出层）

用于实现弹窗、抽屉等交互效果，支持多种弹出方向（上、下、左、右、居中），可自定义弹窗内容，适用于登录弹窗、筛选弹窗、提示弹窗等场景。

```
1   <template>
2     <view class="popup-container">
3       <!-- 触发弹窗按钮 -->
4       <uni-button type="primary" @click="showPopup = true">打开居中弹窗</uni-
button>
5       <uni-button type="default" @click="showBottomPopup = true" style="margin-
top: 20rpx;">打开底部弹窗</uni-button><!-- 居中弹窗 -->
6       <uni-popup v-model="showPopup" mode="center" @close="handleClose">
7         <view class="popup-content">
8           <view class="popup-title">提示</view>
9           <view class="popup-desc">这是一个居中弹出的弹窗</view>
10          <view class="popup-btn">
11            <uni-button type="default" @click="showPopup = false">取消</uni-
button>
12            <uni-button type="primary" @click="showPopup = false">确认</uni-
button>
13          </view>
14        </view>
15      </uni-popup>
16
17      <!-- 底部弹窗 -->
18      <uni-popup v-model="showBottomPopup" mode="bottom">
19        <view class="bottom-popup-content">
20          <view class="popup-item" @click="handleItemClick('选项1')">选项1</view>
21          <view class="popup-item" @click="handleItemClick('选项2')">选项2</view>
22          <view class="popup-item" @click="handleItemClick('选项3')">选项3</view>
23          <uni-button type="default" @click="showBottomPopup = false"
style="margin-top: 20rpx;">取消</uni-button>
24        </view>
25      </uni-popup>
26    </view>
27  </template>
28
29  <script setup>
30    import { ref } from 'vue'
31
32    // 控制弹窗显示/隐藏
```

```
33  const showPopup = ref(false)
34  const showBottomPopup = ref(false)
35
36  // 弹窗关闭事件
37  const handleClose = () => {
38    console.log('弹窗关闭')
39  }
40
41  // 底部弹窗选项点击
42  const handleItemClick = (item) => {
43    uni.showToast({ title: `选择${item}`, icon: 'none' })
44    showBottomPopup.value = false
45  }
46  </script>
47
48  <style scoped>
49    .popup-container {
50      padding: 30rpx;
51    }
52    .popup-content {
53      width: 600rpx;
54      padding: 30rpx;
55      background-color: #fff;
56      border-radius: 20rpx;
57      text-align: center;
58    }
59    .popup-title {
60      font-size: 32rpx;
61      font-weight: bold;
62      margin-bottom: 20rpx;
63    }
64    .popup-desc {
65      font-size: 28rpx;
66      color: #666;
67      margin-bottom: 30rpx;
68    }
69    .popup-btn {
70      display: flex;
71      gap: 20rpx;
72    }
73    .popup-btn uni-button {
74      flex: 1;
75    }
76    .bottom-popup-content {
77      padding: 30rpx;
78      background-color: #fff;
79      border-top-left-radius: 20rpx;
```

```
80     border-top-right-radius: 20rpx;
81   }
82   .popup-item {
83     font-size: 30rpx;
84     padding: 20rpx 0;
85     border-bottom: 1rpx solid #eee;
86     text-align: center;
87   }
88   </style>
```

三、uni-ui 通用使用规范

1. 组件命名：所有 uni-ui 组件均以「uni-」开头（如 uni-button、uni-list），使用时需严格按照官方命名书写，避免拼写错误。
2. 样式自定义：uni-ui 组件支持通过 CSS 变量或内联样式自定义样式，推荐使用组件自带的 CSS 变量（如 --input-height），避免直接覆盖组件内部样式，防止多端适配异常。
3. 多端适配：uni-ui 已做好多端兼容，但部分组件在不同平台仍可能存在细微差异，可通过 uniApp 条件编译（`// #ifdef H5` 等）微调样式，适配不同平台。
4. 组件版本：uni-ui 会持续更新，若遇到组件功能异常，可检查组件版本是否最新，通过插件市场或 NPM 更新组件。
5. 事件绑定：Vue3 setup 中，直接使用 @事件名（如 @click、@change）绑定组件事件，无需添加 .native 修饰符。
6. 按需使用：若仅需使用少数组件，推荐通过 uni_modules 单独安装对应组件，减少项目体积；若需使用多个组件，可全局安装 uni-ui 组件库。

四、补充说明

1. 更多组件：uni-ui 还提供了 uni-calendar（日历）、uni-steps（步骤条）、uni-search-bar（搜索栏）、uni-collapse（折叠面板）等组件，用法与上述核心组件类似，可参考官方文档查询详细属性和示例。
2. 官方文档：若需了解组件完整属性、事件、样式变量，可访问 uniApp 官方文档的 uni-ui 专区，获取最准确的使用指南。
3. 异常排查：若组件无法正常显示，可检查以下几点：① 安装是否成功；② easycom 配置是否正确；③ 组件命名是否正确；④ 依赖（如 Sass）是否安装。

（注：文档部分内容可能由 AI 生成）